

Shipyard Product Requirements Document (PRD)

Version: 1.0

Status: Draft – Approved for Product Backlog Creation

Product: Shipyard

Document Owner: Yonatane Mekete

Last Updated: July 2026

1. Introduction

Purpose

This Product Requirements Document (PRD) defines the functional and non-functional requirements for **Shipyard**, a modern open-source project management platform built specifically for small software engineering teams.

The PRD serves as the primary reference for product management, design, frontend development, backend development, quality assurance, and future contributors. It translates the product vision into clear, implementable requirements that guide the development of the MVP and future releases.

Product Summary

Shipyard enables small engineering teams to plan, organize, collaborate on, and deliver software through a fast, opinionated, and developer-first workflow.

The product centers around five core workflows:

- Managing work
- Planning projects
- Running development cycles
- Collaborating with teammates
- Tracking progress

Rather than maximizing customization, Shipyard emphasizes simplicity, consistency, and excellent defaults, allowing teams to focus on building software instead of configuring their project management tool.

Target Audience

Shipyard is designed primarily for:

- Startup engineering teams (2–30 people)
- Indie hackers
- Solo developers
- Small software agencies

It is not intended to compete with enterprise-focused project management platforms or support highly customized organizational workflows.

Product Principles

Every requirement in this document should align with these principles:

- Simplicity over complexity
- Build for software teams first
- Opinionated by default
- Speed first
- Consistency across workflows
- Quality over quantity

When requirements conflict, these principles should guide decision-making.

MVP Definition

Version 1 is considered complete when a small engineering team can:

- Create a workspace
- Invite team members
- Create and organize projects
- Manage issues
- Plan and complete development cycles

- Collaborate through comments
 - Track team progress
 - Use Shipyard as their primary project management tool
-

Document Scope

This PRD defines:

- Functional requirements
- User stories
- Acceptance criteria
- Business rules
- Permissions
- System behaviors
- MVP boundaries

It does **not** define:

- UI design specifications
 - Database schema
 - API contracts
 - Technical architecture
 - Implementation details
-

2. Product Goals

Purpose

Product goals describe the outcomes Shipyard is designed to achieve. Every feature included in the MVP should contribute directly to one or more of these goals.

Goal 1 — Simplify Software Project Management

Enable small software teams to organize and manage their work without the complexity of enterprise project management tools.

Success Indicators

- Teams can start using Shipyard with minimal onboarding.
 - Common workflows require little to no configuration.
 - Users spend more time building software than managing the tool.
-

Goal 2 — Help Teams Ship Software Faster

Reduce the friction involved in planning, assigning, tracking, and completing development work.

Success Indicators

- Teams can easily prioritize work.
 - Development cycles remain organized.
 - Progress is always visible.
 - Work moves smoothly from backlog to completion.
-

Goal 3 — Deliver an Exceptional Developer Experience

Provide a fast, intuitive, and consistent experience designed specifically for software engineering teams.

Success Indicators

- Fast page loads and interactions.
 - Keyboard-friendly workflows.
 - Consistent UI patterns.
 - Minimal clicks for common actions.
-

Goal 4 — Build a Production-Grade Open Source Platform

Develop Shipyard with production-quality engineering practices while keeping it accessible for contributors and self-hosting.

Success Indicators

- Clear documentation.
 - Easy local development setup.
 - Well-structured architecture.
 - Maintainable codebase.
 - Transparent contribution process.
-

Goal 5 — Establish a Reusable Foundation

Create architecture, patterns, and documentation that can be reused across future Build Stage products.

This is an internal engineering goal, but it's important because Shipyard is the first project in the Build Stage ecosystem.

Success Indicators

- Reusable authentication patterns.
 - Shared UI components.
 - Scalable project structure.
 - Consistent engineering standards.
-

Goal Prioritization

Not all goals have the same priority. If trade-offs arise, we'll prioritize them in this order:

Priorit y	Goal
P1	Simplify software project management
P1	Help teams ship software faster

- P2 Deliver an exceptional developer experience
- P2 Build a production-grade open-source platform
- P3 Establish a reusable foundation

This ordering helps resolve conflicts. For example, if a feature improves developer experience but significantly complicates the product for users, we'd choose the simpler product because our highest priorities are serving small software teams.

Product Success Statement

When Shipyard reaches MVP, we want a small engineering team to be able to say:

"We replaced our project management tool with Shipyard and never felt like we were missing the essentials."

That sentence captures the outcome we're aiming for better than any individual metric.

3. Glossary

Term	Definition
Workspace	The top-level container for a team and its data.

Member	A user who belongs to a workspace.
Issue	A unit of work such as a bug, task, or feature.
Project	A collection of related issues with a shared goal.
Cycle	A fixed development period used to organize work.
Assignee	The member responsible for an issue.
Label	A tag used to categorize issues.
Notification	An in-app alert about relevant activity.

4. User Personas

Persona 1 — Startup Founder / Product Manager

Responsibilities

- Plan product roadmap
- Prioritize work
- Create projects and issues
- Track team progress

Goals

- Keep development organized
- Ensure the team ships on time
- Maintain visibility across projects

Pain Points

- Too much time spent managing tools
- Difficult to see overall progress
- Complex project management software

Primary Actions in Shipyard

- Create projects
 - Create and prioritize issues
 - Plan cycles
 - Review dashboards
 - Monitor team progress
-

Persona 2 — Software Engineer

Responsibilities

- Implement features
- Fix bugs
- Update issue status
- Collaborate with teammates

Goals

- Know what to work on
- Track personal tasks
- Communicate efficiently

Pain Points

- Unclear priorities
- Losing context across discussions
- Switching between multiple tools

Primary Actions in Shipyard

- View assigned issues

- Update issue status
 - Comment on issues
 - Complete tasks
 - Receive notifications
-

Persona 3 — Engineering Lead

Responsibilities

- Coordinate developers
- Plan development cycles
- Remove blockers
- Monitor delivery

Goals

- Balance workload
- Keep projects on schedule
- Identify bottlenecks early

Pain Points

- Limited visibility into team progress
- Difficulty tracking blocked work
- Too much manual coordination

Primary Actions in Shipyard

- Manage cycles
 - Assign issues
 - Track project progress
 - Review dashboards
 - Resolve blockers
-

Why Only These Three?

These personas represent the majority of users in a small software team:

- **Founder / Product Manager** → Decides what gets built.
- **Engineering Lead** → Decides who builds it and when.

- **Software Engineer** → Builds it.

Designing for these three covers nearly every workflow in our MVP.

5. Functional Requirements

Standard Feature Template

Every feature in this section follows the same structure:

- Overview
 - Goals
 - User Stories
 - Functional Requirements
 - Business Rules
 - Acceptance Criteria
 - Edge Cases
 - Future Enhancements (*optional*)
-

5.1 Authentication

Overview

Authentication allows users to securely create an account, sign in, access their workspaces, and manage their sessions.

Authentication is the entry point to Shipyard and must provide a secure, frictionless experience while supporting future workspace collaboration.

Goals

- Secure user accounts.
- Fast onboarding.
- Support individual users and teams.
- Minimize login friction.

User Stories

- As a new user, I want to create an account so I can start using Shipyard.
 - As a returning user, I want to sign in quickly.
 - As a logged-in user, I want to remain signed in across sessions.
 - As a user, I want to sign out securely.
-

Functional Requirements

Registration

- Users can register using an email and password.
- Email must be unique.
- Password must meet minimum security requirements.
- New users are automatically signed in after successful registration.

Login

- Users can log in with their email and password.
- Invalid credentials display a generic error message.
- Successful login redirects users to their workspace selection or dashboard.

Logout

- Users can log out from any device.
- Logging out invalidates the current session.

Session Management

- Sessions persist across browser refreshes.
- Expired sessions redirect users to the login page.
- Protected pages require authentication.

Password Management

- Users can request a password reset.
 - Password reset links expire after a configurable time.
 - Users can change their password after logging in.
-

Business Rules

- Email addresses are unique.
 - Passwords are never stored in plain text.
 - Authentication is required for all protected routes.
 - Users may belong to multiple workspaces.
 - Authentication is separate from workspace permissions.
-

Acceptance Criteria

Registration

Given a new user

When valid registration details are submitted

Then the account is created and the user is signed in.

Login

Given an existing account

When valid credentials are entered

Then access is granted.

Protected Routes

Given an unauthenticated visitor

When they access a protected page

Then they are redirected to the login screen.

Password Reset

Given a registered email

When a password reset is requested

Then a reset email is sent.

Edge Cases

- Duplicate email registration.
 - Invalid email format.
 - Weak password.
 - Expired password reset link.
 - Invalid reset token.
 - Expired session.
 - Attempt to access protected pages without authentication.
-

Future Enhancements

- OAuth (Google, GitHub).
 - Passkeys.
 - Two-factor authentication.
 - Single Sign-On (SSO).
 - Device/session management.
-

5.2 Workspace

Overview

A workspace is the primary container for a team's data. It groups members, projects, issues, cycles, and settings into an isolated environment where a team collaborates.

Goals

- Provide an isolated environment for each team.
 - Allow users to create and join workspaces.
 - Keep workspace data separated.
 - Support collaboration within a team.
-

User Stories

- As a new user, I want to create a workspace so I can start managing my team's work.
 - As a user, I want to join an existing workspace through an invitation.
 - As a member, I want to switch between my workspaces.
 - As a workspace owner, I want to manage workspace information.
-

Functional Requirements

Workspace Creation

- Users can create a new workspace.
- A workspace must have a name.
- A workspace may have an icon or logo.
- The creator becomes the workspace owner.
- A newly created workspace is immediately available for use.

Workspace Management

- Workspace owners can update the workspace name.
- Workspace owners can update the workspace icon.
- Workspace owners can archive the workspace.
- Archived workspaces cannot be actively used.
- Workspace owners can permanently delete the workspace.

Workspace Membership

- Users can belong to multiple workspaces.
- Each workspace maintains its own member list.
- Members only have access to workspaces they belong to.

Workspace Switching

- Users can view all workspaces they belong to.
 - Users can switch between workspaces.
 - Switching workspaces updates all visible data to the selected workspace.
-

Business Rules

- Every project belongs to one workspace.
 - Every issue belongs to one workspace.
 - Every cycle belongs to one workspace.
 - Workspace data is isolated from other workspaces.
 - Deleting a workspace removes all associated data.
 - Only workspace owners can archive or delete a workspace.
-

Acceptance Criteria

Create Workspace

Given an authenticated user

When they create a workspace with valid information

Then the workspace is created and they become its owner.

Join Workspace

Given a valid invitation

When a user accepts it

Then they become a member of the workspace.

Switch Workspace

Given a user belongs to multiple workspaces

When they select a different workspace

Then all displayed data updates to the selected workspace.

Archive Workspace

Given a workspace owner

When they archive the workspace

Then the workspace becomes read-only and is no longer active.

Edge Cases

- Duplicate workspace names.
 - User attempts to access a workspace they are not a member of.
 - Last remaining owner attempts to leave the workspace.
 - Workspace deleted while members are active.
 - Invalid or expired invitation link.
 - Archived workspace accessed directly via URL.
-

Future Enhancements

- Workspace templates.
 - Workspace duplication.
 - Custom branding.
 - Workspace transfer to another owner.
 - Organization-level management.
-

5.3 Members

Overview

The Members feature allows teams to collaborate by inviting users to a workspace, assigning roles, and managing membership.

Goals

- Allow teams to grow through invitations.
 - Control who has access to a workspace.
 - Manage member roles and permissions.
 - Keep workspace membership secure and organized.
-

User Stories

- As a workspace owner, I want to invite teammates.
- As a user, I want to accept an invitation and join a workspace.

- As an owner, I want to change a member's role.
 - As an owner, I want to remove members from my workspace.
 - As a member, I want to leave a workspace.
-

Functional Requirements

Invitations

- Workspace owners and authorized members can invite users.
- Invitations are sent using an email address.
- Users can accept or decline an invitation.
- Invitations expire after a defined period.
- Invitations can be revoked before they are accepted.
- Users cannot accept the same invitation more than once.

Member Management

- Workspace owners can view all members.
- Member information includes name, email, role, and join date.
- Owners can remove members.
- Members can leave a workspace voluntarily.
- Owners can transfer workspace ownership to another member.

Roles

The MVP supports three roles:

Owner

- Full workspace access.
- Manage members.
- Manage workspace settings.
- Delete or archive the workspace.

Admin

- Manage projects.
- Manage issues.
- Invite members.
- Manage cycles.

Member

- View workspace content.
 - Create and update issues.
 - Comment on issues.
 - Participate in projects and cycles.
-

Business Rules

- Every member belongs to a workspace.
 - A user may belong to multiple workspaces.
 - Every workspace must always have at least one owner.
 - Removed members immediately lose access.
 - Pending invitations do not grant access.
 - Users cannot invite themselves.
 - Users already in the workspace cannot be invited again.
-

Acceptance Criteria

Invite Member

Given a user with permission

When they invite another user

Then a pending invitation is created.

Accept Invitation

Given a valid invitation

When it is accepted

Then the user becomes a workspace member.

Remove Member

Given an owner

When they remove a member

Then the member immediately loses workspace access.

Leave Workspace

Given a member

When they leave

Then they are removed from the workspace.

Transfer Ownership

Given a workspace owner

When ownership is transferred

Then the selected member becomes the new owner.

Edge Cases

- Invitation sent to an existing member.
 - Invitation expires before acceptance.
 - Invitation revoked before acceptance.
 - User tries to use an expired invitation.
 - Owner attempts to remove themselves while they are the only owner.
 - Last owner attempts to leave the workspace.
 - Removed member tries to access the workspace using an old link.
-

Future Enhancements

- Custom roles.
 - Permission editor.
 - Team groups.
 - Bulk member invitations.
 - Organization-wide member management.
-

5.4 Dashboard

Overview

The Dashboard is the default landing page within a workspace. It provides users with a personalized overview of their work, team activity, and project progress.

Goals

- Give users immediate visibility into their work.
 - Surface important updates.
 - Reduce time spent navigating the application.
 - Help teams stay aligned.
-

User Stories

- As a member, I want to see my assigned work.
 - As a team lead, I want to monitor team progress.
 - As a user, I want to quickly identify overdue or blocked work.
 - As a user, I want to resume work without searching for it.
-

Functional Requirements

Personal Overview

- Display issues assigned to the current user.
- Display issues created by the current user.
- Display recently viewed issues.
- Highlight overdue issues.

Workspace Overview

- Display active projects.
- Display the current active cycle.
- Display recent workspace activity.
- Display recently completed issues.

Quick Actions

- Create a new issue.
- Create a new project (authorized users only).
- Search for issues or projects.
- View all assigned issues.

Activity Feed

- Show recent issue updates.
 - Show newly created issues.
 - Show completed issues.
 - Show member comments.
 - Show project updates.
-

Business Rules

- Dashboard content is limited to the selected workspace.
 - Users only see information they have permission to access.
 - Personal sections are specific to the logged-in user.
 - Activity is displayed in reverse chronological order.
 - Archived items are excluded unless explicitly requested.
-

Acceptance Criteria

Dashboard Load

Given a logged-in workspace member

When they open the dashboard

Then they see an overview of their work and workspace activity.

Personal Tasks

Given assigned issues

When the dashboard loads

Then those issues appear in the personal overview.

Activity Feed

Given recent workspace activity

When the dashboard loads

Then the latest activity is displayed first.

Quick Actions

Given a user with permission

When they select a quick action

Then the corresponding workflow begins immediately.

Edge Cases

- Workspace with no projects.
 - Workspace with no issues.
 - User has no assigned work.
 - No active development cycle.
 - Empty activity feed.
 - Recently archived items.
-

Future Enhancements

- Customizable dashboard widgets.
 - Team performance insights.
 - Burndown and velocity charts.
 - Calendar summary.
 - Personalized recommendations.
 - AI-generated daily summaries.
-

5.5 Issues

Overview

Issues represent individual units of work within a workspace. They are used to track bugs, features, tasks, improvements, and other development work from creation to completion.

Goals

- Capture all development work.
 - Provide a clear workflow from backlog to completion.
 - Keep issue information organized and searchable.
 - Enable collaboration around work items.
-

User Stories

- As a user, I want to create an issue so work can be tracked.
 - As a user, I want to update an issue as work progresses.
 - As a team lead, I want to assign issues to team members.
 - As a developer, I want to know the priority of my work.
 - As a team member, I want to discuss work within an issue.
-

Functional Requirements

Issue Creation

- Users can create an issue.
- Every issue must have a title.
- Issues may include a description.
- Issues are automatically assigned a unique identifier.
- New issues are created in the **Backlog** by default.

Issue Details

Each issue supports:

- Title
- Description
- Status
- Priority
- Assignee

- Project
- Cycle
- Labels
- Due date
- Creator
- Created date
- Last updated date

Issue Management

Users can:

- Edit issue details.
- Change issue status.
- Assign or reassign issues.
- Add or remove labels.
- Associate an issue with a project.
- Associate an issue with a cycle.
- Set or update a due date.
- Archive an issue.
- Delete an issue (subject to permissions).

Issue Workflow

Issues move through predefined workflow states:

- Backlog
- Todo
- In Progress
- Done

The system records status changes in the issue history.

Issue Relationships

- Every issue belongs to one workspace.
- An issue may belong to one project.
- An issue may belong to one cycle.
- An issue may have one assignee.
- An issue may have multiple labels.

Issue Discovery

Users can:

- Search issues.
 - Filter issues.
 - Sort issues.
 - View issue details.
 - View issue history.
-

Business Rules

- Every issue belongs to exactly one workspace.
 - Every issue has one current status.
 - Archived issues are read-only.
 - Completed issues remain searchable.
 - Issue identifiers are unique within a workspace.
 - Only authorized members can delete issues.
 - Changes to an issue are recorded in its activity history.
-

Acceptance Criteria

Create Issue

Given a workspace member

When they create an issue with a valid title

Then the issue is added to the workspace backlog.

Update Status

Given an existing issue

When its status is changed

Then the new status is saved and recorded in the issue history.

Assign Issue

Given a workspace member

When they assign an issue

Then the assignee is updated successfully.

Archive Issue

Given an existing issue

When it is archived

Then it no longer appears in active issue lists.

Search Issues

Given matching issues exist

When a user performs a search

Then relevant issues are returned.

Edge Cases

- Empty issue title.
 - Extremely long title or description.
 - Deleted assignee.
 - Archived project linked to an issue.
 - Archived cycle linked to an issue.
 - Duplicate labels.
 - Invalid due date.
 - Simultaneous edits by multiple users.
 - Attempt to edit an archived issue.
-

Future Enhancements

- Parent-child issues.
- Issue dependencies.
- Subtasks.
- Custom issue types.
- Recurring issues.
- Issue templates.
- Watchers.
- Estimated effort/story points.
- Rich text editing.
- File attachments.

5.6 Projects

Overview

Projects organize related issues into larger initiatives with a shared objective. They provide teams with visibility into progress, ownership, and timelines for delivering features or milestones.

Goals

- Organize related work into meaningful initiatives.
 - Track progress toward project goals.
 - Provide visibility into project status.
 - Help teams plan and deliver features efficiently.
-

User Stories

- As a product manager, I want to create a project for a new feature.
 - As a team lead, I want to group related issues under a project.
 - As a developer, I want to understand how my work contributes to the project.
 - As a stakeholder, I want to monitor project progress.
-

Functional Requirements

Project Creation

- Users with permission can create a project.
- Every project must have a name.
- Projects may include a description.
- Projects may have a start date.
- Projects may have a target completion date.

- Every project has an owner.

Project Details

Each project supports:

- Name
- Description
- Status
- Owner
- Start date
- Target date
- Progress
- Associated issues
- Created date
- Last updated date

Project Management

Users with permission can:

- Edit project information.
- Archive a project.
- Delete a project.
- Change the project owner.
- Update the project status.

Project Organization

Users can:

- Add issues to a project.
- Remove issues from a project.
- View all issues within a project.
- Search and filter project issues.
- Navigate from a project to any associated issue.

Project Progress

The system should:

- Display the total number of issues.
- Display completed and remaining issues.
- Display overall project progress.
- Update project progress automatically as issue statuses change.

Project Status

Projects support the following statuses:

- Planned
 - Active
 - Completed
 - Archived
-

Business Rules

- Every project belongs to exactly one workspace.
 - A project can contain multiple issues.
 - An issue can belong to only one project.
 - Archived projects are read-only.
 - Deleting a project does not delete its issues.
 - Removing an issue from a project does not delete the issue.
-

Acceptance Criteria

Create Project

Given a user with permission

When they provide the required information

Then a new project is created.

Add Issue to Project

Given an existing project and issue

When the issue is added

Then it appears in the project's issue list.

Track Progress

Given project issues change status

When progress is recalculated

Then the project reflects the updated completion percentage.

Archive Project

Given an active project

When it is archived

Then it becomes read-only and no longer appears in active project lists.

Edge Cases

- Project created without issues.
 - All issues removed from a project.
 - Project owner leaves the workspace.
 - Archived issue remains associated with a project.
 - Project reaches its target date with unfinished issues.
 - Duplicate project names.
 - Attempt to modify an archived project.
-

Future Enhancements

- Project templates.
 - Milestones.
 - Project dependencies.
 - Roadmaps.
 - Project health indicators.
 - Custom project fields.
 - Cross-project reporting.
 - Budget and resource tracking.
-

5.7 Cycles

Overview

Cycles are fixed development periods used to plan, organize, and track work. They help teams focus on a defined set of issues and measure progress toward short-term goals.

Goals

- Organize work into time-boxed iterations.
 - Help teams prioritize deliverables.
 - Improve visibility into sprint progress.
 - Keep teams focused on current work.
-

User Stories

- As a team lead, I want to create a development cycle.
 - As a team lead, I want to assign issues to a cycle.
 - As a developer, I want to know what work belongs to the current cycle.
 - As a team member, I want to track cycle progress.
-

Functional Requirements

Cycle Creation

- Users with permission can create a cycle.
- Every cycle must have a name.
- Every cycle must have a start date.
- Every cycle must have an end date.
- A cycle may include a goal or description.

Cycle Management

Users with permission can:

- Edit cycle details.
- Start a cycle.
- Complete a cycle.
- Archive a cycle.
- Delete a cycle before it starts.

Issue Planning

Users can:

- Add issues to a cycle.
- Remove issues from a cycle.
- Move issues between cycles.
- View all issues assigned to a cycle.

Cycle Progress

The system should:

- Display total issues.
- Display completed issues.
- Display remaining issues.
- Display overall completion progress.
- Update progress automatically as issue statuses change.

Cycle Status

Cycles support the following statuses:

- Planned
- Active
- Completed
- Archived

Only one cycle can be **Active** within a workspace at a time.

Business Rules

- Every cycle belongs to one workspace.
 - An issue can belong to only one cycle at a time.
 - Active cycles cannot overlap within the same workspace.
 - Completed cycles become read-only.
 - Completing a cycle does not automatically complete unfinished issues.
 - Archived cycles remain available for historical reference.
-

Acceptance Criteria

Create Cycle

Given a user with permission

When they provide valid cycle information

Then a new cycle is created.

Assign Issue to Cycle

Given an existing issue

When it is assigned to a cycle

Then it appears in that cycle's issue list.

Start Cycle

Given a planned cycle

When it is started

Then its status changes to **Active**.

Complete Cycle

Given an active cycle

When it is completed

Then its status changes to **Completed** and no further edits are allowed.

Edge Cases

- Cycle created with no issues.
- Cycle reaches its end date before completion.
- Issue moved out of an active cycle.
- Attempt to activate a second cycle while another is active.
- Attempt to edit a completed cycle.
- Cycle deleted while it still contains issues.

Future Enhancements

- Automatic recurring cycles.
 - Carry unfinished issues into the next cycle.
 - Sprint velocity reporting.
 - Burndown charts.
 - Capacity planning.
 - Team workload balancing.
-

5.8 Comments

Overview

Comments allow workspace members to discuss work directly within an issue, keeping conversations organized and attached to the relevant context.

Goals

- Enable collaboration around issues.
 - Keep discussions contextual.
 - Maintain a history of decisions and updates.
 - Reduce the need for external communication tools.
-

User Stories

- As a developer, I want to ask questions about an issue.
 - As a reviewer, I want to provide feedback on an issue.
 - As a team member, I want to mention a colleague to get their attention.
 - As a user, I want to edit or remove my comments if needed.
-

Functional Requirements

Comment Creation

- Members can add comments to an issue.
- Comments support plain text and basic formatting.
- Comments are displayed in chronological order.
- Every comment displays its author and creation time.

Comment Management

Users can:

- Edit their own comments.
- Delete their own comments.
- View the edit history indicator for modified comments.

Users with appropriate permissions can remove any comment.

Mentions

- Users can mention workspace members using @.
- Mentioned users receive a notification.
- Mentions link to the referenced member.

Comment History

- Comments remain associated with their issue.
 - Deleted comments are removed from the conversation.
 - Edited comments display that they have been modified.
-

Business Rules

- Comments belong to exactly one issue.
 - Only workspace members can comment.
 - Archived issues cannot receive new comments.
 - Users can only edit or delete their own comments unless they have elevated permissions.
 - Comments are ordered chronologically.
-

Acceptance Criteria

Add Comment

Given an active issue

When a member submits a comment

Then the comment appears in the issue discussion.

Edit Comment

Given an existing comment

When its author edits it

Then the updated content is displayed and marked as edited.

Mention User

Given a workspace member

When they are mentioned in a comment

Then they receive a notification.

Archived Issue

Given an archived issue

When a user attempts to comment

Then the action is prevented.

Edge Cases

- Empty comment submission.
- Very long comments.
- Mentioning a non-existent user.
- Mentioning a user who has left the workspace.
- Editing a deleted comment.
- Simultaneous edits by multiple users.

Future Enhancements

- Emoji reactions.

- Rich text editor.
 - File attachments.
 - Code blocks with syntax highlighting.
 - Comment threads and replies.
 - Comment pinning.
-

5.9 Notifications

Overview

Notifications inform users about relevant activity within their workspace, helping them stay updated without constantly checking the application.

Goals

- Keep users informed about important events.
 - Surface actions that require attention.
 - Reduce the need to manually monitor issues and projects.
 - Support timely collaboration.
-

User Stories

- As a user, I want to know when I'm assigned an issue.
 - As a user, I want to know when someone mentions me.
 - As a project owner, I want to know about important project updates.
 - As a workspace member, I want to review notifications I've missed.
-

Functional Requirements

Notification Types

The system generates notifications for:

- Issue assignment.
- Issue unassignment.
- Issue status changes (when relevant).
- Mentions in comments.
- Workspace invitations.
- Project ownership changes.
- Cycle assignment or removal.

Notification Center

Users can:

- View all notifications.
- View unread notifications.
- Mark a notification as read.
- Mark all notifications as read.
- Navigate directly to the related item.

Notification Status

Each notification includes:

- Type
- Message
- Related item
- Timestamp
- Read/Unread status

Notification Management

Users can:

- Mark notifications as read.
- Mark all notifications as read.
- Delete individual notifications.
- Clear all notifications.

Business Rules

- Notifications are private to the recipient.
- Notifications belong to one user.
- Read notifications remain accessible until deleted.
- Deleted notifications cannot be restored.

- Opening a notification navigates to the related resource when applicable.
-

Acceptance Criteria

Receive Notification

Given a relevant event occurs

When the event is processed

Then the affected user receives a notification.

Mark as Read

Given an unread notification

When the user marks it as read

Then its status changes to read.

Open Notification

Given a notification references an existing resource

When the user selects it

Then they are taken to the related page.

Clear Notifications

Given existing notifications

When the user clears them

Then they are removed from the notification list.

Edge Cases

- Related issue or project is deleted.
- User is mentioned multiple times in the same comment.
- Notification references an archived resource.
- Large number of unread notifications.

- Duplicate events occurring in quick succession.
-

Future Enhancements

- Email notifications.
 - Push notifications.
 - Notification preferences.
 - Notification grouping.
 - Notification snoozing.
 - Real-time desktop notifications.
 - Digest summaries.
-

5.10 Search & Filters

Overview

Search and Filters enable users to quickly locate issues, projects, and other workspace resources by searching, filtering, and sorting data.

Goals

- Help users find information quickly.
 - Reduce time spent navigating the workspace.
 - Support common engineering workflows.
 - Make large workspaces easy to manage.
-

User Stories

- As a developer, I want to quickly find an issue by its title.
- As a team lead, I want to filter issues by status and assignee.
- As a product manager, I want to view only issues related to a project.
- As a user, I want to sort results to focus on what matters.

Functional Requirements

Search

Users can search for:

- Issues
- Projects
- Cycles
- Members

Search should match relevant information such as names, titles, and descriptions.

Filtering

Users can filter issues by:

- Status
- Priority
- Assignee
- Project
- Cycle
- Labels
- Creator
- Due date

Users can apply multiple filters simultaneously.

Sorting

Users can sort results by:

- Created date
- Updated date
- Due date
- Priority
- Title

Users can change the sort order between ascending and descending where applicable.

Saved Views

Users can:

- Save frequently used filter combinations.
- Rename saved views.
- Delete saved views.
- Set a saved view as the default.

Search Results

The system should:

- Display matching results.
 - Display the number of matching results.
 - Show a message when no results are found.
 - Update results whenever search or filter criteria change.
-

Business Rules

- Search only returns data from the current workspace.
 - Users only see resources they have permission to access.
 - Multiple filters are combined together.
 - Saved views are private to the user.
-

Acceptance Criteria

Search

Given matching resources exist

When a user enters a search query

Then relevant results are displayed.

Apply Filters

Given available filters

When one or more filters are applied

Then only matching results are shown.

Save View

Given an active filter configuration

When the user saves it

Then the saved view is available for future use.

No Results

Given no matching resources exist

When a search or filter is applied

Then an appropriate empty state is displayed.

Edge Cases

- Empty search query.
 - Search returns no results.
 - Invalid filter combinations.
 - Saved view references deleted resources.
 - Filtering archived resources.
 - Very large result sets.
-

Future Enhancements

- Global search across all workspaces.
 - Natural language search.
 - Recent searches.
 - Search suggestions.
 - Advanced query syntax.
 - Shared saved views.
 - Search history.
-

5.11 Settings

Overview

Settings allow users and workspace administrators to manage account preferences, workspace configuration, and team settings.

Goals

- Provide a central place for configuration.
 - Allow users to manage their personal preferences.
 - Allow workspace administrators to manage workspace settings.
 - Keep administrative tasks organized and accessible.
-

User Stories

- As a user, I want to update my profile information.
 - As a user, I want to change my password.
 - As a workspace owner, I want to manage workspace settings.
 - As a workspace owner, I want to manage member roles.
-

Functional Requirements

Profile Settings

Users can:

- Update their display name.
- Update their profile picture.
- Update their email address.
- Change their password.
- View their account information.

Workspace Settings

Workspace owners can:

- Update the workspace name.
- Update the workspace icon.

- Archive the workspace.
- Delete the workspace.
- View workspace information.

Member Management

Authorized users can:

- View workspace members.
- Change member roles.
- Remove members.
- Transfer workspace ownership.

Preferences

Users can:

- Select a theme (Light, Dark, System).
- Configure language (future-ready).
- Manage notification preferences.
- View active sessions.

Danger Zone

Workspace owners can:

- Archive the workspace.
- Permanently delete the workspace.

These actions require confirmation before completion.

Business Rules

- Users can only modify their own account settings.
 - Only workspace owners can modify workspace settings.
 - Only authorized users can manage members.
 - Every workspace must always have at least one owner.
 - Destructive actions require confirmation.
-

Acceptance Criteria

Update Profile

Given a logged-in user

When they save valid profile changes

Then the updated information is reflected throughout the workspace.

Change Password

Given a logged-in user

When they provide valid credentials

Then the password is updated successfully.

Update Workspace

Given a workspace owner

When they modify workspace information

Then the changes are visible to all workspace members.

Delete Workspace

Given a workspace owner

When they confirm deletion

Then the workspace and its associated data are permanently removed.

Edge Cases

- Duplicate workspace name.
- Invalid email address.
- User attempts to change restricted settings.
- Last owner attempts to leave the workspace.
- Workspace deletion while members are active.
- Attempt to perform destructive actions without confirmation.

Future Enhancements

- Custom branding.
 - Workspace domains.
 - API keys.
 - Webhooks.
 - Integration management.
 - Data export.
 - Data import.
 - Audit history.
 - Billing and subscription settings.
-

6. Permissions & Roles

Overview

Shipyards uses role-based access control (RBAC) to determine what actions members can perform within a workspace.

The MVP includes three roles:

- **Owner** – Full administrative access.
 - **Admin** – Manages day-to-day team operations.
 - **Member** – Collaborates on projects and issues.
-

Permission Matrix

Permission	Owner	Admin	Member
View workspace	✓	✓	✓
Update workspace	✓	✗	✗

Archive workspace	✓	✗	✗
Delete workspace	✓	✗	✗
Invite members	✓	✓	✗
Remove members	✓	✓	✗
Change member roles	✓	✗	✗
Transfer ownership	✓	✗	✗
Create projects	✓	✓	✗
Edit projects	✓	✓	✓
Archive projects	✓	✓	✗
Delete projects	✓	✓	✗
Create issues	✓	✓	✓
Edit issues	✓	✓	✓
Archive issues	✓	✓	✓

Delete issues	✓	✓	✗
Create cycles	✓	✓	✗
Manage cycles	✓	✓	✗
Comment on issues	✓	✓	✓
View dashboard	✓	✓	✓
Search workspace	✓	✓	✓
Manage workspace settings	✓	✗	✗

Role Definitions

Owner

Responsible for the workspace and its administration.

Can:

- Manage workspace settings.
- Manage members.
- Transfer ownership.
- Archive or delete the workspace.
- Perform all Admin and Member actions.

Admin

Responsible for managing development work.

Can:

- Manage projects.
- Manage issues.
- Manage cycles.
- Invite and remove members.
- Perform all Member actions.

Cannot:

- Delete the workspace.
 - Modify workspace settings.
 - Change member roles.
 - Transfer ownership.
-

Member

Responsible for contributing to the team's work.

Can:

- Create and update issues.
- Contribute to projects.
- Comment on issues.
- View dashboards.
- Search workspace data.

Cannot:

- Create projects.
 - Manage workspace administration.
 - Manage members.
 - Delete projects.
 - Create or manage cycles.
-

Business Rules

- Every workspace must always have at least one Owner.
- Permissions are evaluated within each workspace.
- Users may have different roles in different workspaces.

- Unauthorized actions are rejected.
 - Role changes take effect immediately.
 - Feature-specific permissions follow this workspace permission matrix unless explicitly stated otherwise.
-

Acceptance Criteria

Permission Check

Given a member attempts an action

When they have the required permission

Then the action succeeds.

Unauthorized Action

Given a member attempts an action

When they lack permission

Then the action is denied.

Edge Cases

- Last owner attempts to leave the workspace.
 - Role changed while the user is active.
 - Removed member attempts to perform an action.
 - Workspace ownership transferred during an active session.
-

Future Enhancements

- Custom roles.
- Fine-grained permissions.
- Permission groups.
- Organization-level roles.
- Resource-specific permissions.

7. Business Rules

Overview

Business Rules define the constraints and behaviors that govern how Shipyard operates. These rules apply across the product and ensure consistent behavior regardless of where an action is performed.

7.1 Workspace Rules

- Every workspace must have at least one Owner.
 - A user may belong to multiple workspaces.
 - User roles are assigned independently for each workspace.
 - All projects, issues, cycles, and members belong to exactly one workspace.
 - Users can only access workspaces they are members of.
 - Deleting a workspace permanently removes all associated data.
-

7.2 Member Rules

- Every member has exactly one role within a workspace.
 - A member can only hold one role at a time.
 - Role changes take effect immediately.
 - Removed members immediately lose access to the workspace.
 - The last remaining Owner cannot leave or be removed until ownership is transferred.
-

7.3 Project Rules

- Every project belongs to exactly one workspace.
- Projects may contain zero or more issues.
- Members can contribute to projects but cannot create or delete them.

- Archived projects become read-only.
 - Deleting a project does not delete its issues.
-

7.4 Issue Rules

- Every issue belongs to exactly one workspace.
 - Every issue may belong to one project.
 - Every issue may belong to one cycle.
 - Every issue has exactly one status.
 - Every issue has exactly one creator.
 - Every issue may have one assignee.
 - Issues retain their history after updates.
 - Deleted issues cannot be recovered.
-

7.5 Cycle Rules

- Every cycle belongs to exactly one workspace.
 - Only one cycle may be active within a workspace at any time.
 - An issue may belong to only one cycle.
 - Completing a cycle does not automatically move unfinished issues.
 - Archived cycles become read-only.
-

7.6 Comment Rules

- Every comment belongs to one issue.
 - Every comment has one author.
 - Users may edit or delete only their own comments.
 - Mentioning a user generates a notification.
-

7.7 Notification Rules

- Notifications are private to the recipient.
- Notifications belong to one user.
- Notifications remain available until deleted.
- Opening a notification does not remove it.

- Marking a notification as read only changes its status.
-

7.8 Search Rules

- Search is limited to the current workspace.
 - Users only see resources they have permission to access.
 - Multiple filters are combined when applied together.
 - Saved views are private to the user.
-

7.9 General Rules

- Archived resources cannot be modified.
 - Unauthorized actions are rejected.
 - System timestamps are recorded automatically.
 - All user-generated content is associated with its creator.
 - Destructive actions require explicit confirmation.
 - Changes should be reflected consistently across the workspace.
-

8. Non-Functional Requirements

Overview

Non-Functional Requirements define the quality attributes of Shipyard. These requirements ensure the product is secure, reliable, performant, accessible, and scalable while providing a consistent user experience.

8.1 Performance

Requirements

- The application should feel responsive during normal use.
- Most user interactions should provide immediate visual feedback.

- Search and filtering should return results quickly.
 - Navigation between pages should be smooth.
 - Large workspaces should remain usable without noticeable slowdowns.
 - Background operations should not interrupt the user experience.
-

8.2 Reliability

Requirements

- The application should handle unexpected failures gracefully.
 - User data should remain consistent after operations.
 - Failed actions should display clear error messages.
 - Users should be able to safely retry failed operations.
 - The system should prevent duplicate actions caused by repeated user input.
-

8.3 Security

Requirements

- Users must be authenticated before accessing protected resources.
 - Authorization must be enforced for all protected actions.
 - Users may only access data belonging to workspaces they are members of.
 - Sensitive user information must be protected.
 - Destructive actions require explicit confirmation.
 - User sessions should be managed securely.
-

8.4 Scalability

Requirements

- The system should support growth in users, workspaces, and projects.
 - Performance should remain acceptable as data volume increases.
 - The product should support future feature expansion without major redesign.
 - The architecture should accommodate additional integrations and modules.
-

8.5 Availability

Requirements

- The application should be available whenever users need it.
 - Temporary service interruptions should be communicated clearly.
 - Users should receive meaningful feedback when services are unavailable.
-

8.6 Usability

Requirements

- The interface should be simple and intuitive.
 - Common actions should require minimal effort.
 - Navigation should remain consistent across the product.
 - Empty states should guide users toward the next action.
 - Error messages should clearly explain what went wrong.
 - Confirmation messages should acknowledge successful actions.
-

8.7 Accessibility

Requirements

- The application should be navigable using a keyboard.
 - Interactive elements should be clearly distinguishable.
 - Color should not be the only indicator of information.
 - Text should remain readable across supported themes.
 - Interface components should support assistive technologies where applicable.
-

8.8 Compatibility

Requirements

- The application should support modern desktop browsers.
- The interface should adapt to common screen sizes.
- Users should have a consistent experience across supported browsers.

- The product should remain functional on tablets, with mobile support considered in future releases.
-

8.9 Maintainability

Requirements

- Product behavior should remain consistent as new features are added.
 - Features should be modular to simplify future development.
 - Documentation should remain aligned with product behavior.
 - Changes to one feature should minimize unintended effects on others.
-

8.10 Observability

Requirements

- System errors should be recorded for investigation.
 - Significant user actions should be traceable where appropriate.
 - Application health should be monitorable.
 - Operational issues should be diagnosable without impacting users.
-

8.11 Data Integrity

Requirements

- Data should remain accurate and consistent after operations.
 - Relationships between resources should remain valid.
 - Invalid or incomplete data should not be accepted.
 - Operations should avoid creating duplicate or conflicting records.
-

Acceptance Criteria

Performance

Given normal application usage

When users perform common actions

Then the application remains responsive and usable.

Security

Given an unauthorized user

When they attempt to access protected resources

Then access is denied.

Reliability

Given an operation fails

When the failure occurs

Then the user receives clear feedback and data remains consistent.

Accessibility

Given a user navigates without a mouse

When they interact with the application

Then all primary functionality remains accessible.

Future Enhancements

- Offline support.
- Progressive Web App (PWA) capabilities.
- Multi-region deployment.
- Advanced monitoring and analytics.
- Localization and internationalization.
- Configurable performance and retention policies.

9. Release Scope

Overview

This section defines the functionality planned for the initial release (MVP), identifies features deferred to future releases, and clarifies what is outside the scope of Shipyard.

9.1 MVP Scope (Version 1.0)

The initial release focuses on providing a complete project management experience for small software teams.

Authentication

- User registration
- User login
- Logout
- Password management
- Session management

Workspace Management

- Create workspace
- Update workspace
- Invite members
- Member roles
- Workspace switching

Issue Management

- Create issues
- Update issues
- Delete issues
- Assign issues
- Issue priorities
- Issue statuses
- Labels
- Due dates
- Search and filtering

Project Management

- Create projects

- Update projects
- Archive projects
- Track project progress

Cycle Management

- Create cycles
- Manage cycles
- Assign issues to cycles
- Cycle progress tracking

Collaboration

- Comments
- User mentions
- Notifications

Dashboard

- Personal overview
- Workspace overview
- Activity feed
- Quick actions

Settings

- Profile management
- Workspace settings
- Member management
- User preferences

9.2 Post-MVP Scope

These features are planned for future releases but are not part of Version 1.0.

Planning

- Kanban Board
- Timeline View
- Calendar View
- Roadmaps
- Milestones

Integrations

- GitHub
- GitLab
- Slack
- Discord
- Email notifications

Productivity

- Saved templates
- Recurring issues
- Attachments
- Rich text editor
- Advanced search
- Shared saved views

Analytics

- Team reports
- Productivity insights
- Velocity tracking
- Burndown charts

Customization

- Custom workflows
 - Custom issue types
 - Custom fields
 - Automation rules
-

9.3 Out of Scope

The following capabilities are intentionally excluded from Shipyard's MVP.

Enterprise Features

- Organization hierarchy
- Multiple organizations
- SSO
- SCIM provisioning
- Advanced audit logs

- Compliance management

Business Features

- CRM
- HR management
- Finance
- Accounting
- Billing management

AI Features

- AI issue generation
- AI sprint planning
- AI summaries
- AI assistants

Advanced Platform Features

- Plugin marketplace
 - Public APIs
 - Workflow automation engine
 - Multi-region administration
-

9.4 Release Goals

Version 1.0 will be considered successful if users can:

- Create and manage workspaces.
 - Collaborate with team members.
 - Organize work using issues, projects, and cycles.
 - Track progress from planning to completion.
 - Receive notifications about important events.
 - Find work quickly through search and filters.
 - Complete everyday project management tasks without requiring external tools.
-

9.5 Success Criteria

The MVP is complete when:

- All MVP functional requirements are implemented.
 - Core user workflows are fully functional.
 - Role-based permissions are enforced.
 - Non-functional requirements are satisfied.
 - The application is stable enough for daily use by small software teams.
-

10. Open Questions & Assumptions

Overview

This section captures assumptions made during product planning and identifies questions that may require validation through user feedback, design exploration, or future product decisions.

10.1 Assumptions

The following assumptions guided the MVP definition:

Target Users

- Shipyard is designed primarily for software teams of **2–30 members**.
 - Teams prefer simplicity over extensive customization.
 - Most users are familiar with modern project management tools.
-

Team Structure

- Every workspace has at least one Owner.
 - Teams have a relatively flat organizational structure.
 - Projects are managed by Owners or Admins.
 - Members primarily contribute by working on issues.
-

Workflow

- Teams organize work using Issues, Projects, and Cycles.
 - Only one active cycle exists per workspace.
 - Notifications help users stay informed but are not intended to replace communication tools.
 - Search is a primary method of navigating large workspaces.
-

Product Philosophy

- Opinionated defaults are preferred over extensive configuration.
 - The MVP prioritizes speed, clarity, and ease of use.
 - Every feature should solve a common software development workflow.
-

10.2 Open Questions

The following decisions are intentionally deferred until future validation.

Workflow

- Should archived resources be restorable?
 - Should completed cycles generate automatic summary reports?
 - Should users be allowed to watch issues they are not assigned to?
-

Collaboration

- Should emoji reactions be supported on comments?
 - Should threaded discussions be introduced?
 - Should users receive digest notifications in addition to real-time notifications?
-

Planning

- What is the best Kanban experience for Shipyard?
 - Should Roadmaps become a core feature or remain optional?
 - Should Milestones be separate entities or part of Projects?
-

Integrations

- Which third-party integration should be prioritized first?
 - Should GitHub integration be one-way or two-way?
 - Which notification channels should be supported beyond the in-app notification center?
-

10.3 Deferred Decisions

The following topics are intentionally postponed until after the MVP:

- Custom workflows.
 - Custom issue types.
 - Custom fields.
 - Automation rules.
 - API access.
 - Plugin ecosystem.
 - Organization support.
 - Billing and subscriptions.
 - AI-powered features.
 - Advanced reporting and analytics.
-

10.4 Risks

Potential risks that may affect the product:

- Scope creep during MVP development.
 - Feature parity expectations with established competitors.
 - Performance challenges as workspace size grows.
 - Balancing simplicity with flexibility.
 - Prioritizing future features without direct user feedback.
-

10.5 Validation Plan

After the MVP is available, the team should gather feedback on:

- Overall usability.
- Issue management workflow.
- Project planning workflow.
- Cycle management.
- Notification usefulness.
- Search experience.
- Team collaboration.
- Missing features that impact daily workflows.